# EXPERIMENT 02

## PCA and LDA

Python implementation using Breast Cancer dataset

```
# Experiment 02: PCA and LDA

# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

print("Original Dataset Shape:", X.shape)

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Logistic Regression on Original Dataset
model = LogisticRegression(max_iter=5000)

start = time.time()
model.fit(X_train, y_train)
original_time = time.time() - start

y_pred = model.predict(X_test)
original_accuracy = accuracy_score(y_test, y_pred)

# 6. Apply PCA (30 features to 10)
pca = PCA(n_components=10)

X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

model_pca = LogisticRegression(max_iter=5000)

start = time.time()
model_pca.fit(X_train_pca, y_train)
pca_time = time.time() - start

y_pred_pca = model_pca.predict(X_test_pca)
pca_accuracy = accuracy_score(y_test, y_pred_pca)

print("\nPCA Reduced Shape:", X_train_pca.shape)

# 7. Apply LDA (30 features to 1)
lda = LinearDiscriminantAnalysis(n_components=1)

X_train_lda = lda.fit_transform(X_train, y_train)
X_test_lda = lda.transform(X_test)

model_lda = LogisticRegression(max_iter=5000)

start = time.time()
model_lda.fit(X_train_lda, y_train)
```

```
lda_time = time.time() - start

y_pred_lda = model_lda.predict(X_test_lda)
lda_accuracy = accuracy_score(y_test, y_pred_lda)

print("LDA Reduced Shape:", X_train_lda.shape)

# 8. Compare Accuracy and Training Time
result = pd.DataFrame({
    'Technique': ['Original', 'PCA', 'LDA'],
    'Features': [30, 10, 1],
    'Accuracy': [original_accuracy, pca_accuracy, lda_accuracy],
    'Training Time': [original_time, pca_time, lda_time]
})

print("\nComparison Results:")
print(result)

# 9. PCA 2D Graph
pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_train)

plt.figure(figsize=(6, 4))
plt.scatter(X_pca_2d[:, 0], X_pca_2d[:, 1],
            c=y_train, cmap='viridis')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA 2D Visualization")
plt.show()

# 10. LDA 2D Graph
plt.figure(figsize=(6, 4))
plt.scatter(X_train_lda[:, 0], y_train,
            c=y_train, cmap='viridis')
plt.xlabel("LDA Component 1")
plt.ylabel("Class (0 = Malignant, 1 = Benign)")
plt.title("LDA 2D Visualization")
plt.show()

print("\nPCA and LDA Completed Successfully!")
```